SBA301 – SLOT 26

CHAPTER 16 – SECURING REST SERVICES WITH SPRING BOOT

PART C: SECURITY CONSOLIDATION • AUTHN/AUTHZ • ROLE • JWT • CORS • NEGATIVE TESTS

CORE: Tư duy trung tâm của Slot 26: Một hệ thống "có JWT" chưa đồng nghĩa là an toàn. Security chỉ được xem là hoàn chỉnh ở mức course khi identity được xác thực đúng, quyền được kiểm soát đúng, CORS hoạt động đúng với browser client, token invalid/expired/tampered bị từ chối đúng, 401/403 phân biệt rõ, và toàn bộ contract được chứng minh bằng negative tests có thể lặp lại.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	25
Chủ đề	Chapter 16 Part C – Security consolidation: authn/authz, role, JWT, CORS, negative tests
CLO chính	CLO4 – Apply/verify REST security integration in Spring Boot applications
Ngữ liệu xuyên suốt	Orchid Management / Lab 06 security baseline + React client integration awareness
Course baseline	JDK 21 • Spring Boot 3.x • Spring Security 6.x style • Spring Data JPA • SQL Server • Postman
Thời lượng gợi ý	135 phút trên lớp + security verification/self-study
Learning flow	Prepare → Consolidate → Trace → Test → Break → Diagnose → Verify → Transfer

NOTE: Scope lock: Slot 26 không dạy lại toàn bộ OAuth/JWT từ đầu. Đây là buổi hợp nhất và kiểm chứng. Sinh viên phải nối được authn → authz → role/authority → JWT → CORS → endpoint policy → negative test matrix. Các nội dung production như Resource Server, issuer/audience validation, JWK rotation, opaque token và DPoP chỉ dùng làm production/current-ecosystem direction.

CÁCH SỬ DỤNG GIÁO TRÌNH

Phần IV là knowledge base tổng hợp để sinh viên xây dựng mental model thống nhất. Phần VI là security verification workbook: từng bước đều có thao tác, code/config, lệnh chạy, expected result, PASS/FAIL và evidence. Mục tiêu cuối buổi không phải 'login được', mà là chứng minh security contract bằng test dương và test âm.

Pha	Sinh viên làm gì?	Evidence
Prepare	Chạy baseline backend, login, protected endpoint và React origin.	Baseline evidence
Consolidate	Lập security map endpoint/role/token/CORS.	Security contract matrix
Trace	Trace login và protected request qua filter chain.	Execution trace
Test	Chạy positive + negative test matrix.	Postman/MockMvc evidence
Break	Tamper JWT, wrong role, wrong origin, missing token.	Failure evidence
Diagnose	Phân loại 401/403/CORS/preflight/token validation failure.	Debug note
Verify	Đối chiếu status/body/header/log/principal.	Verification checklist
Transfer	Đánh giá lab security vs production Resource Server.	Architecture worksheet

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức cần ôn lại

- Slot 24: authentication vs authorization, Spring Security filter chain, JWT anatomy/claims, 401 vs 403.
- REST CRUD từ Slot 22 và React/Axios/CORS từ Slot 23.
- Authorization header: `Bearer <token>`; HTTP OPTIONS preflight ở browser.
- Role/authority: ROLE_USER, ROLE_ADMIN; hasRole vs hasAuthority ở mức conceptual.
- Password hashing và UserDetails/UserDetailsService.
- CORS là browser policy; Postman không mô phỏng đầy đủ browser CORS behavior.

0.2. Công cụ và baseline

Thành phần	Yêu cầu	Kiểm tra
Backend	Spring Boot security baseline chạy được	mvn test / GET public endpoint
DB	User + Role data tồn tại	SSMS query
Postman	Environment có base_url/access_token	login → token
Browser/React	Origin localhost:5173 hoặc origin khóa học	DevTools Network
IDE	IntelliJ + debugger	breakpoint filter/controller
Git	checkpoint sạch	git status

Thành phần	Yêu cầu	Kiểm tra

0.3. Pre-class 3–2–1

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi cần chứng minh
JWT có signature; role quyết định quyền; CORS ảnh hưởng browser.	401/403 đôi khi bị lẫn; preflight có cần token không?	Một request từ React với Bearer token + CORS đi qua những lớp nào và lỗi ở đâu nếu bị chặn?

0.4. Pre-flight checklist

- Login hợp lệ trả access token.
- Protected endpoint không token trả 401.
- Có ít nhất USER và ADMIN để test role.
- React/Browser client chạy ở origin khác backend.
- Không log raw token/password.
- Commit/tag gợi ý: s26-before-security-consolidation.

Baseline commands

```
mvn -q -DskipTests compile
mvn test
mvn spring-boot:run

# Browser client (nếu dùng)
npm install
npm run dev
```

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Security consolidation map

Boundary	Câu hỏi	Control	Evidence
Identity	Caller là ai?	Authentication	Principal
Credential	Chứng minh identity bằng gì?	Password/Bearer token	Auth header / login
Authorization	Caller được làm gì?	Role/authority/policy	200/403
Token integrity	Token có bị sửa?	Signature + claims validation	tampered token → 401
Browser origin	Frontend có được gọi API?	CORS	preflight/headers
Failure contract	Từ chối như thế nào?	401/403/CORS error	negative tests

1.2. Mục tiêu học tập

1. Hợp nhất authn/authz/JWT/role/CORS thành một security request lifecycle duy nhất.
2. Thiết kế endpoint security matrix public/authenticated/admin.
3. Phân biệt role và authority; giải thích mapping `ROLE_ADMIN` và policy.
4. Kiểm chứng JWT signature/expiration/subject/role claims ở mức course.
5. Giải thích CORS preflight và vì sao nó phải được xử lý phù hợp với Spring Security.
6. Thiết kế negative test matrix cho endpoint protected.
7. Phân biệt chính xác 401, 403 và browser CORS failure.
8. Trace request từ React/Postman qua CORS → SecurityFilterChain → JWT → authorization → Controller.
9. Chẩn đoán misconfiguration: permitAll quá rộng, role prefix sai, filter order sai, origin sai.
10. Chuyển tư duy lab sang production Resource Server/JWT validation chuẩn hóa.

1.3. Mental model end-to-end

```
React / Browser (origin http://localhost:5173)
  |
  |-- OPTIONS preflight? ---------------------------+
  |                                                 |
  |                                CORS processing  |
  |                                                 v
```

```
+-- actual request + Authorization: Bearer <JWT>
        |
        v
   Spring Security Filter Chain
        |
        +-- Bearer/JWT authentication
        |     - parse/verify token
        |     - validate exp/claims
        |     - build Authentication
        |
        +-- SecurityContext
        |
        +-- Authorization rule
        |     - permitAll?
        |     - authenticated?
        |     - role/authority?
        |
        +-- allow --------------------> Controller
        |
        +-- reject authn ------------> 401
        |
        +-- reject authz ------------> 403
```

```
Browser may still block response if CORS headers are not valid.
```

KEY: CORS failure ≠ 401/403. CORS là browser enforcement; authentication/authorization là server security decisions.

1.4. Definition of Done cho Slot 26

Không đủ	Đủ course-ready
Login trả token	Login + token + protected access + role policy + negative tests
Có @CrossOrigin('*')	Specific origin/method/header policy + browser verification
USER/ADMIN test thủ công 1 lần	Repeatable matrix: no token / valid / wrong role / tampered / expired
API trả lỗi bất kỳ	401/403 đúng semantics; CORS lỗi được phân biệt
JWT decode được	JWT phải verify + validate claims

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. Authentication pipeline

Credential được chuyển thành Authentication request; provider/user service/password/token validation xác lập principal. Authentication thành công không tự động cấp mọi quyền.

2.2. Authorization pipeline

Authorization dùng Authentication + authorities + request/resource context để quyết định allow/deny. URL rule chỉ là một lớp; domain ownership có thể cần business authorization.

2.3. Role vs Authority

Role thường là semantic grouping và Spring convention thường map `hasRole('ADMIN')` thành authority `ROLE_ADMIN`. `hasAuthority('ADMIN')` và `hasRole('ADMIN')` không tương đương nếu naming khác.

2.4. JWT validation

Không chỉ decode. Tối thiểu phải kiểm signature + expiration; production profile còn thường kiểm issuer/audience và các claims bắt buộc.

2.5. Bearer token behavior

Bearer token là credential. Ai có token hợp lệ có thể dùng nó; vì vậy HTTPS, storage và log hygiene là bắt buộc.

2.6. CORS lifecycle

Browser có thể gửi preflight OPTIONS trước actual request. Preflight thường không mang Bearer token/user credential theo cùng cách actual request, nên CORS phải được xử lý trước khi security từ chối request.

2.7. CORS policy components

Allowed origins, methods, headers, credentials, exposed headers và max age. Wildcard không phải mặc định đúng; origin phải càng cụ thể càng tốt.

2.8. 401 vs 403 vs CORS

401: chưa xác thực/credential invalid. 403: đã authenticated nhưng không đủ quyền. CORS: browser không cho JavaScript đọc/gửi request theo policy.

2.9. Negative testing mindset

Test security phải cố tình gửi request sai. Positive-only test không chứng minh access control.

2.10. Token tampering test

Sửa payload/signature phải bị từ chối. Nếu hệ thống chỉ base64-decode rồi tin claims, đó là critical design error.

2.11. Expired token test

Token quá hạn phải bị reject. Không kéo expiration rất dài chỉ để test 'đỡ lỗi'.

2.12. Missing token test

Protected endpoint không Bearer token phải trả 401, không 200/500.

2.13. Wrong-role test

Authenticated USER gọi ADMIN endpoint phải 403.

2.14. Public endpoint test

Public endpoint phải truy cập được không token nhưng không nên vô tình mở protected endpoint.

2.15. Preflight test

OPTIONS từ allowed origin phải nhận CORS headers phù hợp; denied origin phải không được cấp quyền cross-origin.

2.16. Defense against configuration drift

Security matrix phải trở thành automated regression tests; nếu endpoint mới được thêm mà quên policy, test phải phát hiện.

2.17. Security outcome decision table

Scenario	Authentication	Authorization	CORS	Expected
Public GET, no token	N/A	permitAll	allowed origin	200
Protected GET, no token	fail/missing	not evaluated	allowed origin	401
Protected GET, valid USER	success	allow authenticated	allowed origin	200
ADMIN endpoint, valid USER	success	deny role	allowed origin	403
ADMIN endpoint, valid ADMIN	success	allow	allowed origin	200
Protected GET, tampered JWT	fail	N/A	allowed origin	401
Protected GET, expired JWT	fail	N/A	allowed origin	401
Valid token from disallowed browser origin	server auth may succeed	may allow	browser denies	CORS failure in browser

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Security matrix – Orchid Management

Endpoint	Access	Role	Negative tests bắt buộc
POST /auth/login	Public	N/A	wrong password, unknown user
GET /orchids	Public hoặc authenticated theo contract	N/A	CORS denied origin
POST /orchids	Protected	USER/STAFF	no token, tampered, expired

Endpoint	Access	Role	Negative tests bắt buộc
PUT /orchids/{id}	Protected	USER/STAFF	wrong role, invalid id
DELETE /orchids/{id}	Protected	ADMIN direction	USER→403, no token→401
GET /admin/users	Protected	ADMIN	USER→403, tampered→401

3.2. FUNewsManagement transfer

Trong Assignment 02, Account/Category/NewsArticle có role và business constraint. Slot 26 dùng security consolidation để buộc sinh viên phân biệt access-control rule (Admin/Staff) với business rule (ví dụ không xóa Category đang được tham chiếu). Security role cho phép caller đi vào use case; business rule vẫn phải được Service kiểm tra.

Rule	Security layer	Business layer
Chỉ Admin quản lý Account	hasRole/authority	validation/account rules
Staff quản lý Category	role access	không xóa category đang được dùng
Staff quản lý News	authenticated staff	ownership/history/data rule
Public đọc active news	permitAll GET	query chỉ NewsStatus=active

3.3. Production consolidation direction

Current Spring Security hỗ trợ OAuth2 Resource Server với JWT hoặc opaque bearer token. Với JWT, framework có thể xác minh signature, exp/nbf/iss và map scopes thành authorities. Điều này giảm lượng custom token parsing/filter code so với lab hand-rolled JWT. CORS vẫn cần cấu hình riêng và phải được xử lý đúng trước Spring Security.

Course custom JWT	Production direction
Custom JwtService + filter	OAuth2 Resource Server JwtDecoder
Shared secret	Issuer/JWK/public key; key rotation
Manual role claim mapping	JwtAuthenticationConverter / scope-authority mapping
Manual parser exception	framework bearer-token error handling
Manual CORS filter/config	Spring MVC/Security CorsConfigurationSource
Manual negative Postman only	MockMvc/WebTest + API security matrix

IV. NỘI DUNG CHI TIẾT – KNOWLEDGE BASE SECURITY CONSOLIDATION

READING GUIDE: Phần IV là phần lý thuyết hệ thống hóa. Không trình bày như lab. Sinh viên dùng phần này để giải thích 'tại sao' và phân loại lỗi trước khi làm Phần VI.

4.1. Security contract như một truth table

- Mỗi endpoint phải có ít nhất: method/path, public/protected, required role/authority, expected 401/403, CORS policy và business precondition.
- Security matrix tốt giúp test, code review và tránh endpoint mới bị bỏ quên policy.

4.2. Authentication state machine

- Unauthenticated → credential supplied → validation → authenticated principal hoặc authentication failure.
- JWT request không 'login lại' bằng password; token là credential của request và phải được verify mỗi request theo security model.

4.3. Authorization state machine

- Sau authentication, policy evaluate authorities/roles/scopes + request/resource context.
- Authorization failure sau authenticated phải khác authentication failure.

4.4. Role/authority semantics trong Spring Security

- `hasRole('ADMIN')` thường thêm prefix `ROLE_`; `hasAuthority('ROLE_ADMIN')` kiểm authority trực tiếp.
- Role naming mismatch là lỗi thực tế phổ biến: token claim='ADMIN' nhưng Authentication authorities='ROLE_ADMIN' hoặc ngược lại.

4.5. JWT integrity vs confidentiality

- Signature/MAC đảm bảo token không bị sửa mà không phát hiện.
- Payload không mặc định encrypted; không để secret/password/sensitive personal data trong claims chỉ vì token 'đã ký'.

4.6. JWT claim trust boundary

- Claim chỉ đáng tin sau signature/key/algorithm validation.

- Role claim phải đến từ issuer/server-controlled source, không lấy role client gửi lúc login rồi ký mù quáng.

4.7. exp/nbf/iat và clock

- Expiration cần negative test rõ. `nbf` có thể làm token chưa hợp lệ. Clock skew cần có policy.
- Token lifetime là risk-control decision, không phải con số tiện tay.

4.8. iss/aud ở production

- Issuer xác định ai phát token; audience xác định token dành cho service nào.
- OWASP khuyến nghị validate iss/aud/exp cho JWT API access trong profile phù hợp.

4.9. CORS request model

- Origin = scheme + host + port. `http://localhost:5173` và `http://localhost:3000` là origin khác.
- Simple request và preflighted request có behavior khác; Authorization/custom headers thường làm browser gửi OPTIONS preflight.

4.10. CORS trước Spring Security

- Spring Security documentation nhấn mạnh preflight phải được xử lý trước security vì preflight không mang session cookie như actual request.
- Trong bearer-token SPA, tư duy tương tự: đừng buộc OPTIONS phải có user JWT.

4.11. Allowed origins/methods/headers

- AllowedOrigins phải cụ thể theo environment; allowedMethods phải phản ánh API; allowedHeaders thường cần Content-Type, Authorization.
- ExposeHeaders chỉ cần khi JavaScript phải đọc response headers ngoài safelist.

4.12. allowCredentials và wildcard

- Credentialed CORS cần policy chặt; wildcard origin không dùng tùy tiện với credentials.
- Bearer token trong Authorization header vẫn là sensitive credential; CORS policy không thay thế authn/authz.

4.13. Browser vs Postman

- Postman/curl không enforce browser CORS. 'Postman chạy' không chứng minh React browser chạy.
- Security QA phải có cả server tests và browser DevTools khi scope có CORS.

4.14. Negative testing taxonomy

- Credential negative: missing, malformed, tampered, expired.
- Authorization negative: wrong role/scope/ownership.
- Origin negative: disallowed Origin/preflight method/header.
- Input/business negative: invalid body/not found/conflict — không được nhầm với security failure.

4.15. JWT tampering patterns

- Mutate claim value nhưng giữ signature cũ → reject.
- Remove/change signature → reject.
- Change algorithm/header must not cause verifier to accept insecure algorithm based solely on token header.

4.16. Broken access control tests

- Object-level: user A truy cập resource của user B.
- Function-level: USER gọi admin function.
- Field-level: caller sửa privileged field như role/status/owner trong request body.

4.17. Public endpoint exposure

- permitAll chỉ dành endpoint thực sự public. Rule ordering/matcher rộng có thể vô tình mở endpoint.
- Regression test phải kiểm endpoint protected vẫn 401/403 khi cấu hình thay đổi.

4.18. Security exception translation

- AuthenticationEntryPoint xử lý unauthenticated; AccessDeniedHandler xử lý access denied.
- Không để JWT parser exception tràn thành 500.

4.19. CORS error interpretation

- Browser console thường nói blocked by CORS, nhưng network request có thể đã tới server hoặc bị preflight chặn.
- DevTools cần xem OPTIONS status/headers và actual request có được gửi hay không.

4.20. Logging security failures

- Có thể log category/outcome nhưng không log raw password/token.
- 401 spike, 403 spike, tampered token spike có thể trở thành detection signal.

4.21. Test layering

- Unit: token service/role mapper.
- Web security slice/integration: matcher, 401/403, CORS.
- Full integration: DB user + login + token + protected endpoint.
- Browser test: CORS/SPA integration.

4.22. MockMvc security tests

- Dùng request không token, mock user/real token tùy boundary mục tiêu.
- Không dùng mock user để 'chứng minh JWT parser' vì nó bỏ qua token path.

4.23. API security test matrix

- Mỗi operation nên test no credential, valid credential, insufficient privilege.
- OWASP REST Assessment khuyến nghị xây matrix theo security requirement và fuzz/tamper token trước khi chỉ test business endpoint.

4.24. CORS negative test matrix

- Allowed origin + allowed method/header → pass.
- Disallowed origin → no allow-origin header / browser blocks.
- Allowed origin nhưng disallowed method/header → preflight fail.

4.25. Security vs business failures

- 401/403 là security boundary; 400 validation; 404 resource; 409 business conflict.
- Không dùng 403 cho mọi business rule vì sẽ làm client và observability sai nghĩa.

4.26. Role changes và JWT staleness

- Nếu role nằm trong JWT, role DB thay đổi có thể chưa có hiệu lực tới khi token hết hạn nếu service không re-check.
- Đây là trade-off self-contained token vs introspection/stateful checks.

4.27. Logout/revocation awareness

- Xóa token client không thu hồi token đã copy.
- Production cần short lifetime, rotation, denylist/introspection tùy threat model.

4.28. Security consolidation anti-patterns

- `permitAll('/**')` để sửa lỗi 401.
- `@CrossOrigin('*')` trên mọi controller.
- JWT secret hard-code.
- Token expiration cực dài.
- Role lấy trực tiếp từ request body khi signup.
- Catch mọi exception rồi trả 200 message.

• Chỉ test happy path.

4.29. Resource Server current direction

• Spring Security current stable docs hỗ trợ JWT và opaque bearer tokens.
• JWT Resource Server có thể verify signature/exp/nbf/iss và map scopes thành `SCOPE_...` authorities.
• Spring Boot hỗ trợ issuer-uri, jwk-set-uri và audiences property ở current ecosystem.

4.30. Production security review checklist

• Endpoint inventory, trust boundary, authn source, authz policy, token validation, CORS, secrets, TLS, negative tests, logs/metrics, incident readiness.
• Security review phải kiểm evidence chứ không chỉ annotation presence.

V. KINH NGHIỆM THỰC TIỄN, TIPS VÀ DEBUGGING

Lỗi	Triệu chứng	Phân loại	Cách xử lý
Missing token	401	Authn	Kiểm endpoint protected và Authorization header
Malformed/tampered token	401 hoặc 500 nếu code lỗi	Authn	Parser/verify + exception mapping
USER gọi ADMIN	403	Authz	Authorities + matcher/method security
Role prefix mismatch	403 dù 'đúng role'	Authz config	ROLE_ mapping
OPTIONS bị 401	React báo CORS	CORS/order	CORS trước security; allow OPTIONS/preflight
Wrong origin	Postman OK, browser fail	CORS	AllowedOrigins exact
Authorization header not allowed	Preflight fail	CORS	AllowedHeaders include Authorization
permitAll quá rộng	API protected thành public	Config	Narrow matchers + regression test
JWT expired	401	Authn	exp/clock
Parser exception → 500	500	Error handling	map invalid token to 401
Raw token in logs	Credential leakage	Operational security	mask/remove
Cross-role business action	200 nhưng sai quyền	Broken access control	role/ownership test

5.1. Debugging decision tree

```
React request fails
|
+-- Browser shows CORS?
|   +-- Inspect OPTIONS preflight
|   +-- Access-Control-Allow-Origin?
|   +-- Allowed method?
|   +-- Authorization/Content-Type allowed?
|   +-- Was actual request sent?
|
+-- Actual request reached backend
    |
    +-- 401?
    |   +-- Header missing/malformed?
    |   +-- token tampered/expired?
    |   +-- signature/key/claims invalid?
    |
    +-- 403?
    |   +-- Authentication exists?
    |   +-- authorities?
    |   +-- hasRole/hasAuthority mismatch?
    |   +-- ownership/policy?
    |
    +-- 2xx but security should deny?
        +-- matcher too broad?
        +-- permitAll ordering?
        +-- method security missing?
```

5.2. Practical tips

• Viết security matrix trước khi code test.
• Khi debug CORS, luôn kiểm OPTIONS và actual request riêng.

- Không dùng Postman để kết luận CORS đã đúng.
- Negative test cần repeatable, không chỉ thao tác thủ công.
- Mỗi protected endpoint tối thiểu phải có no-token + insufficient-role test.
- Nếu custom JWT filter có nhiều code parsing/claims/error mapping, ghi rõ production migration direction sang Resource Server.
- Không để CORS workaround (`*`) che lỗi architecture.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

WORKBOOK RULE: Mỗi bước: Mục tiêu → thao tác → code/config → command → expected result → PASS/FAIL evidence.

6.1. Step 1 – Chụp baseline security contract

Endpoint	Method	Expected access	Role	No token	Wrong role
/auth/login	POST	Public	-	200/401 by credential	-
/orchids	GET	Public	-	200	-
/orchids	POST	Protected	USER/STAFF	401	403 nếu role không đủ
/admin/users	GET	Protected	ADMIN	401	403 USER

PASS: toàn lớp thống nhất contract trước khi chạy test.

6.2. Step 2 – Xác nhận JWT positive path

Login

```
POST {{base_url}}/auth/login
Content-Type: application/json

{
  "email": "student24@example.com",
  "password": "Sba301@2026"
}
```

Lưu access token vào Postman environment variable `access_token`.

```
GET {{base_url}}/users/me
Authorization: Bearer {{access_token}}
```

PASS: 200 và identity/authorities đúng user.

6.3. Step 3 – Missing token negative test

```
GET {{base_url}}/users/me
# deliberately no Authorization header
```

Expected: 401. FAIL nếu 200 hoặc 500.

EVIDENCE: Lưu screenshot status + response headers/body.

6.4. Step 4 – Tampered JWT test

Copy token, sửa một ký tự ở payload hoặc signature nhưng không re-sign.

```
GET {{base_url}}/users/me
Authorization: Bearer <tampered-token>
```

Expected: 401. Nếu 200, implementation có lỗi nghiêm trọng vì chỉ decode/token trust mà không verify.

6.5. Step 5 – Expired token test

Tạm cấu hình expiration rất ngắn trong test environment (ví dụ 5–10 giây), login lấy token, chờ hết hạn rồi gọi protected endpoint.

```
security.jwt.expiration-ms=5000
```

Expected: token mới 200; sau expiry → 401. Sau test trả lại expiration bình thường.

SECURITY: Không dùng token vô hạn để né lỗi.

6.6. Step 6 – Role/authority test

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/orchids/**").permitAll()
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()
);
```

Test USER token gọi `/admin/users` → 403; ADMIN token → 200.

DIAGNOSE: Nếu USER nhận 401 thay vì 403, kiểm xem authentication có thực sự thành công không.

6.7. Step 7 – Kiểm tra role prefix

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
System.out.println(auth.getAuthorities()); // chỉ dùng local debug, không production logging bừa bãi
```

Quan sát authority thực tế: `ROLE_USER` hay `USER`. Đối chiếu `hasRole`/`hasAuthority`.

PASS: policy và authority naming nhất quán.

6.8. Step 8 – Cấu hình CORS consolidation

Course-ready CORS source

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration c = new CorsConfiguration();
    c.setAllowedOrigins(List.of("http://localhost:5173"));
    c.setAllowedMethods(List.of("GET","POST","PUT","DELETE","OPTIONS"));
    c.setAllowedHeaders(List.of("Authorization","Content-Type"));
    c.setExposedHeaders(List.of("Location"));
    c.setAllowCredentials(false);

    UrlBasedCorsConfigurationSource source =
        new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", c);
    return source;
}

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .cors(cors -> {})
        .csrf(csrf -> csrf.disable())
        // authorization...
        ;
    return http.build();
}
```

NOTE: Nếu project dùng cookie credentials thì allowCredentials/threat model khác; không copy `false`/`true` máy móc.

6.9. Step 9 – Manual preflight verification bằng curl/CMD

Windows CMD

```
curl -i -X OPTIONS http://localhost:8080/orchids ^
  -H "Origin: http://localhost:5173" ^
  -H "Access-Control-Request-Method: POST" ^
  -H "Access-Control-Request-Headers: authorization,content-type"
```

Expected: response có `Access-Control-Allow-Origin: http://localhost:5173` và method/header phù hợp.

FAIL nếu preflight trả 401 hoặc không có CORS headers cần thiết.

6.10. Step 10 – Disallowed origin negative test

```
curl -i -X OPTIONS http://localhost:8080/orchids ^
  -H "Origin: http://evil.example" ^
  -H "Access-Control-Request-Method: POST" ^
  -H "Access-Control-Request-Headers: authorization,content-type"
```

Expected: không cấp allow-origin cho `http://evil.example`.

VERIFY: Server response code có thể khác theo config; tiêu chí chính là browser origin không được authorize qua CORS headers.

6.11. Step 11 – Browser DevTools verification

- Mở React tại `http://localhost:5173`.
- DevTools → Network → thực hiện request protected.
- Quan sát OPTIONS trước actual request nếu có.
- Kiểm Request Headers: Origin, Access-Control-Request-*, Authorization.
- Kiểm Response Headers: Access-Control-Allow-Origin, Allow-Methods/Headers.
- Xác nhận actual request chỉ chạy sau preflight pass.

PASS: Browser hoạt động đúng; không chỉ Postman.

6.12. Step 12 – Negative test: wrong allowed header

Tạm bỏ `Authorization` khỏi allowedHeaders rồi chạy lại React protected request.

Expected: preflight fail/CORS browser error. Khôi phục config sau khi chụp evidence.

6.13. Step 13 – Negative test: permitAll regression

Trong branch/test local, cố ý cấu hình `requestMatchers('/**').permitAll()` trước rule protected.

Expected: negative test `meWithoutToken_returns401` FAIL, chứng minh regression suite bắt lỗi.

BREAK-IT: Không merge cấu hình lỗi. Mục tiêu là chứng minh test có giá trị.

6.14. Step 14 – MockMvc security regression tests

Authorization boundary tests

```
@SpringBootTest
@AutoConfigureMockMvc
class SecurityConsolidationTests {

    @Autowired MockMvc mvc;

    @Test
    void protectedEndpoint_withoutToken_returns401() throws Exception {
        mvc.perform(get("/users/me"))
           .andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "USER")
    void adminEndpoint_withUserRole_returns403() throws Exception {
        mvc.perform(get("/admin/users"))
           .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void adminEndpoint_withAdminRole_returns200() throws Exception {
        mvc.perform(get("/admin/users"))
           .andExpect(status().isOk());
```

```
    }
}
```

TESTING: Mock user tests authorization nhưng không test JWT parser. Phải có ít nhất một integration test dùng token thật để kiểm JWT path.

6.15. Step 15 – CORS MockMvc test

```
@Test
void preflight_allowedOrigin_hasCorsHeaders() throws Exception {
    mvc.perform(options("/orchids")
        .header("Origin", "http://localhost:5173")
        .header("Access-Control-Request-Method", "POST")
        .header("Access-Control-Request-Headers", "authorization,content-type"))
       .andExpect(header().string(
           "Access-Control-Allow-Origin",
           "http://localhost:5173"));
}
```

Bổ sung test disallowed origin không nhận allow-origin tương ứng.

6.16. Step 16 – JWT real-path integration test

Tạo user test → POST login → lấy token từ JSON → gọi protected endpoint với Bearer token. Đây là test chứng minh authentication path thật, khác với `@WithMockUser`.

```
// Pseudocode flow
String token =
    login("student24@example.com", "Sba301@2026");

mvc.perform(get("/users/me")
    .header("Authorization", "Bearer " + token))
   .andExpect(status().isOk());
```

6.17. Step 17 – Complete negative test matrix

ID	Scenario	Expected
N01	Protected/no token	401
N02	Malformed token	401
N03	Tampered signature	401
N04	Expired token	401
N05	Valid USER → ADMIN endpoint	403
N06	Valid ADMIN → ADMIN endpoint	200
N07	Public endpoint/no token	200
N08	Allowed origin preflight	CORS headers present
N09	Disallowed origin preflight	No authorization for origin
N10	Disallowed requested header	Preflight fail
N11	Wrong role prefix/config	Regression detected
N12	JWT parser exception	401, not 500
N13	Login bad password	401
N14	Unknown user	401 / agreed auth contract
N15	Business conflict after valid auth	409/contract, not 403

6.18. Step 18 – Evidence package

- Security endpoint matrix.
- Postman collection + environment (không chứa secret/token thật khi nộp).
- Screenshots/exports cho 401,403,tampered,expired.
- Browser DevTools evidence cho allowed/disallowed CORS.
- Automated test report.
- Execution trace login + protected request.
- Break-It → Diagnose → Fix note.
- README security limitations + production migration note.

6.19. Integration Gate G1–G14

Gate	Evidence
G1 Baseline	build/test/start pass
G2 Authn	valid login + invalid login
G3 JWT	tampered/expired rejected
G4 Principal	identity/authorities correct
G5 Authz	USER/ADMIN boundary correct
G6 401	missing/invalid credential
G7 403	authenticated but insufficient role
G8 CORS allow	allowed origin browser works
G9 CORS deny	disallowed origin blocked
G10 Preflight	OPTIONS not rejected as auth failure
G11 Public contract	public endpoint remains public
G12 Tests	negative matrix automated/manual evidence
G13 Trace	filter/authz/controller chain
G14 Explain	human verification: CORS ≠ authn ≠ authz

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ Slot 26
Authentication (Authn)	Xác lập caller identity	valid JWT → principal
Authorization (Authz)	Quyết định quyền truy cập	USER vs ADMIN
Role	Nhóm quyền mang semantic business	ADMIN
Authority	Permission string Spring Security kiểm	ROLE_ADMIN
Principal	Identity đã authenticated	email/sub
SecurityContext	Chứa Authentication hiện tại	SecurityContextHolder
Bearer Token	Credential dạng bearer	Authorization header
JWT	Token format có claims + signature	access token
Claim	Thuộc tính trong JWT	sub/exp/role
Tampering	Sửa token trái phép	payload mutation
Expiration	Thời hạn token	exp
401	Authentication missing/invalid	no token
403	Authenticated nhưng bị deny	wrong role
CORS	Browser cross-origin policy	5173 → 8080
Origin	scheme + host + port	http://localhost:5173
Preflight	OPTIONS kiểm cross-origin request	before POST with Authorization
AllowedOrigin	Origin được server chấp nhận	localhost:5173
AllowedHeader	Header browser được phép gửi	Authorization
Access-Control-Allow-Origin	Response CORS header	allowed origin
Negative Test	Test cố ý request sai	expired token
Broken Access Control	Caller truy cập ngoài quyền	USER → admin API
Resource Server	API bảo vệ bearer token	production direction
JwtDecoder	Spring component verify/decode JWT	Resource Server
Issuer	Token issuer	iss
Audience	Token target	aud

VIII. RESEARCH BASE VÀ AI-BASED LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

8.1. Quy tắc AI

- Không paste password, private key, JWT secret hoặc token thật.
- Execution claim phải verify bằng source/debugger/log/test.
- CORS claim phải kiểm bằng browser/OPTIONS evidence.
- Current security recommendation phải kiểm official docs/RFC/OWASP.
- AI không được đổi security contract hoặc 'fix' bằng permitAll/CORS wildcard mà không giải thích risk.

8.2. Nhóm I – Workflow / execution rule

PROMPT I.1: Trace một React POST protected request có Authorization header từ browser preflight đến controller. Tách OPTIONS và actual request.

PROMPT I.2: Trace missing-token request qua SecurityFilterChain và xác định nơi tạo 401.

PROMPT I.3: Trace valid USER token vào ADMIN endpoint và xác định nơi tạo 403.

PROMPT I.4: Trace JWT tampering case từ token parser/decoder đến AuthenticationEntryPoint.

PROMPT I.5: Phân tích role mapping từ DB/token claim đến GrantedAuthority và hasRole.

PROMPT I.6: Lập call chain cho login → token → protected request trong source tôi cung cấp.

PROMPT I.7: Phân tích SecurityContext lifetime trong stateless bearer request.

PROMPT I.8: Trace CORS allowed origin và disallowed origin khác nhau ở đâu.

PROMPT I.9: Phân tích rule order trong authorizeHttpRequests và tìm matcher có thể mở endpoint.

PROMPT I.10: Tạo 15 breakpoint/log points để chứng minh toàn bộ security lifecycle.

8.3. Nhóm II – 10 prompt kỹ thuật nâng cao

PROMPT II.1: Thiết kế exhaustive negative JWT matrix: malformed segments, bad signature, expired, nbf future, wrong issuer, wrong audience, wrong role claim. Giải thích expected 401/403.

PROMPT II.2: Phân tích hasRole vs hasAuthority và role prefix. Tạo 6 ví dụ cấu hình sai gây 403 giả.

PROMPT II.3: Phân tích CORS preflight với Authorization header và Content-Type application/json. Liệt kê chính xác request/response headers cần quan sát.

PROMPT II.4: So sánh `@CrossOrigin` per-controller với global CorsConfigurationSource trong hệ thống nhiều API. Phân tích governance trade-off.

PROMPT II.5: Thiết kế object-level authorization test: user A đọc/sửa resource user B. Chỉ ra vì sao role test chưa đủ.

PROMPT II.6: Phân tích stale role trong self-contained JWT khi admin đổi role DB giữa token lifetime. So sánh giải pháp.

PROMPT II.7: Phân tích JWT algorithm/key confusion và vì sao verifier không nên tin thuật toán do token tự chỉ định mà không policy.

PROMPT II.8: Thiết kế fuzz tests cho Authorization header: missing Bearer, extra spaces, duplicate headers, huge token, invalid base64url.

PROMPT II.9: Phân tích race/security issue khi SecurityContext bị set sai hoặc reused; giải thích thread/request scope.

PROMPT II.10: Review một SecurityFilterChain + CORS config và tìm 20 anti-patterns có thể dẫn đến broken access control hoặc CORS bypass/misbehavior.

8.4. Nhóm III – 10 prompt production engineering

PROMPT III.1: Chuyển custom JWT app sang Spring OAuth2 Resource Server. Lập migration map class-by-class: cái gì bỏ, cái gì giữ, test nào giữ nguyên.

PROMPT III.2: Thiết kế security test strategy cho CI/CD: unit, MockMvc, token integration, browser CORS, DAST/API fuzzing, release gate.

PROMPT III.3: Thiết kế centralized endpoint-security inventory từ OpenAPI và effective security requirements; sinh negative matrix tự động.

PROMPT III.4: Thiết kế audit logging cho authn/authz failures mà không rò token/PII. Chọn fields và retention.

PROMPT III.5: Thiết kế key rotation incident drill: key compromise, rotate, invalidate, monitor, rollback.

PROMPT III.6: Thiết kế multi-service role/scope model cho Product/Order/Payment; tránh dùng một ADMIN role toàn hệ thống.

PROMPT III.7: Thiết kế environment-specific CORS dev/staging/prod và cách tránh commit localhost wildcard vào production.

PROMPT III.8: Thiết kế observability dashboard cho 401/403/CORS/preflight/token-validation failures và phân tích false positives.

PROMPT III.9: Threat-model SPA token storage + CORS + XSS + CSRF trong ba phương án storage; chọn kiến trúc và trade-off.

PROMPT III.10: Thiết kế API gateway + downstream Resource Server strategy: nơi validate token, nơi enforce fine-grained authorization, cách tránh trust-by-header.

8.5. Nhóm IV – 10 prompt current ecosystem / community trends

PROMPT IV.1: Kiểm tra Spring Security stable hiện tại và tài liệu OAuth2 Resource Server: JWT vs opaque tokens, dependencies, issuer-uri, jwk-set-uri.

PROMPT IV.2: Kiểm tra current Spring Security CORS guidance và giải thích vì sao CORS phải xử lý trước security/preflight.

PROMPT IV.3: Đọc OWASP REST Security Cheat Sheet phần Access Control/JWT/CORS và tạo checklist mapping vào Slot 26.

PROMPT IV.4: Đọc OWASP REST Assessment Cheat Sheet và tạo negative test matrix từ effective OpenAPI security requirements.

PROMPT IV.5: Kiểm tra Spring Boot current properties cho Resource Server audience validation và giải thích khi nào aud bắt buộc.

PROMPT IV.6: Khảo sát current Spring Authorization Server/OAuth2.1 support và phân loại nội dung nào course nên biết nhưng chưa cần triển khai.

PROMPT IV.7: Khảo sát DPoP support hiện tại trong Spring Security và giải thích replay problem mà sender-constrained token giải quyết.

PROMPT IV.8: So sánh JWT vs opaque token theo current Spring docs/OWASP: revocation, privacy, latency, operational complexity.

PROMPT IV.9: Khảo sát API security testing tools/community practice hiện tại cho JWT tampering, access control, CORS và OpenAPI-driven fuzzing; ưu tiên official/project docs.

PROMPT IV.10: Tạo '2026 Spring REST Security Consolidation Radar': authn, authz, JWT, Resource Server, CORS, negative testing, DPoP, mTLS, passkeys, policy engines; ghi evidence/date.

8.6. AI Verification Worksheet

Claim	Evidence source	Experiment	Conclusion
CORS preflight order	Spring docs + browser	OPTIONS DevTools	
JWT rejected when tampered	source/test	mutate token	
USER→ADMIN is 403	security test	role matrix	
Resource Server current behavior	official Spring docs	minimal sample	
OWASP negative matrix	OWASP + OpenAPI	run cases	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

□ 1. Tôi phân biệt authn, authz, role, authority và principal.
□ 2. Tôi có thể giải thích request nào phải 401 và request nào phải 403.
□ 3. Tôi có thể chứng minh token tampered/expired bị reject.
□ 4. Tôi có thể giải thích hasRole và ROLE_ prefix.
□ 5. Tôi hiểu CORS là browser policy, không phải authentication.
□ 6. Tôi có thể trace preflight OPTIONS và actual request riêng.
□ 7. Tôi biết vì sao Postman chạy không chứng minh browser CORS đúng.
□ 8. Tôi có thể viết negative test matrix cho một endpoint.
□ 9. Tôi có thể phân biệt security failure với validation/business failure.
□ 10. Tôi có thể phát hiện permitAll quá rộng bằng automated test.
□ 11. Tôi có thể kiểm CORS allowed/disallowed origin bằng curl và browser.
□ 12. Tôi có thể nói rõ limitation của custom JWT lab.
□ 13. Tôi hiểu production direction Resource Server/JWT/opaque token.
□ 14. Tôi có evidence 401,403,tampered,expired,CORS allow/deny.
□ 15. Tôi có thể review một security config dựa trên contract thay vì annotation presence.

IX.1. Exit ticket

Câu hỏi	Trả lời
Valid USER token gọi ADMIN endpoint → status?	
Browser CORS failure có luôn nghĩa backend trả 401 không?	
JWT payload bị sửa nhưng signature cũ phải thế nào?	
Preflight request có mục tiêu gì?	
Negative test quan trọng nhất bạn đã tự chạy hôm nay?	

TÀI LIỆU THAM CHIẾU & SOURCE TRACEABILITY

- Course source: Chapter 16 – Securing REST Services with Spring Boot: security mechanisms, Spring Security and JWT implementation.
- Course source: Lab 06 – Spring Boot 3 + SQL Server + Spring Data JPA + JWT registration/login.

- Official Spring Security: CORS integration – CORS must be processed before Spring Security for preflight scenarios.
- Official Spring Security: OAuth2 Resource Server – current support for JWT and opaque bearer tokens.
- Official Spring Boot: Resource Server properties including issuer-uri, jwk-set-uri and audiences.
- OWASP REST Security Cheat Sheet – access control, JWT validation, CORS and HTTPS.
- OWASP REST Assessment Cheat Sheet – per-operation security test matrix and JWT tampering tests.

NOTE: Current ecosystem note (verified October 2026): Spring Security documentation identifies 7.1.1 as current stable in the retrieved reference pages. The course still uses Spring Boot 3.x / Spring Security 6.x style as baseline; version-specific production recommendations must be verified before adoption.